SE2020 – Web and Mobile Technologies

BSc (Hons) in Software Engineering

2nd Year

Faculty of Computing

SLIIT

2026 – Lab 09 (Self-Guided)

NoSQL and MongoDB

What you will learn in this lab sheet

In this lab, you will design and implement a **Node.js + Express + MongoDB backend** for a small real-world scenario.

- Design multiple related MongoDB collections.
- Implement RESTful endpoints using Express and Mongoose.
- Use searching and pagination on API endpoints.
- Implement analytics-style endpoints using the MongoDB aggregation pipeline.
- Test all endpoints using Postman.

This lab builds on the CRUD examples with Express and Mongoose already covered in lectures.

1. MongoDB

1.1 What is MongoDB

MongoDB is a popular open-source NoSQL database. Unlike traditional relational databases that use tables and rows, MongoDB is document-oriented and stores data in JSON-like documents with optional schemas. MongoDB stores data in BSON (Binary JSON) format, which extends the JSON model to provide additional data types and efficient encoding and decoding across different programming languages.

1.2 Work with MongoDB

1.2.1 Setup

1. Download MongoDB Community Server
    - URL: https://www.mongodb.com/try/download/community
2. Open MongoDB Compass
    - Launch MongoDB Compass and wait for the connection screen.

3. Connect to MongoDB
    - Connect to the default local MongoDB instance or use a MongoDB Atlas connection URI if provided.

```
mongodb+srv://wmt_labsheet_12:<db_password>@cluster0.mk29wik.mongodb.net/?
appName=Cluster0
```

Replace **<db_password>** with the password for the **wmt_labsheet_12** database user. Ensure any option params are URL encoded .

2. Scenario – Campus Food Ordering API

The university has "Computing Building Canteen", which serves food to students.

There is **no frontend** for this lab. All interactions will be tested using **Postman**.

2.1 System Requirements

The system must support:

1. Managing student information.
2. Managing menu items available in the campus canteen.
3. Allowing students to place food orders with multiple menu items.
4. Tracking order status (PLACED, PREPARING, DELIVERED, CANCELLED).
5. Providing summary information such as:
    1. Total amount spent by a student.
    2. Most popular menu items.
    3. Daily order counts.

3. Project Setup (Node.js + Express + MongoDB)

Follow these steps in order.

3.1 Create the Project Folder

1. Create a folder named campus-food-api.
2. Open the folder in your code editor.

3.2 Initialize the Node.js Project

1. Use npm to initialize the Node.js project and create a package.json file.
2. Fill in basic project details as required.

3.3 Install Required Dependencies

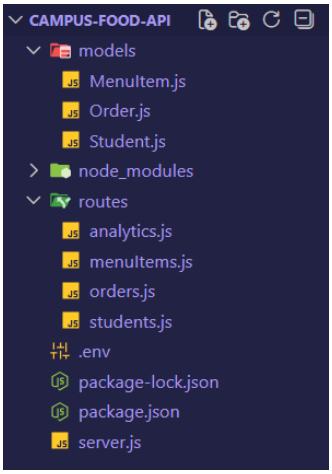
1. Install the following packages:
    1. express – web framework

2. mongoose – ODM for MongoDB

2. Confirm the dependencies appear in package.json.

3.4 Create Base File Structure

1. In the project folder, create:
    1. server.js – main entry point
    2. A models folder
    3. A routes folder
2. Inside models, create:
    1. Student.js
    2. MenuItem.js
    3. Order.js
3. Inside routes, create:
    1. students.js
    2. menuItems.js
    3. orders.js
    4. analytics.js



3.5 Configure the Express Server

1. In server.js:
    1. Create an Express application.
    2. Add middleware to parse JSON request bodies.
    3. Create a basic root route (GET /) to test server status.

```
// server.js
// TODO: Load environment variables from .env file

const express = require('express');
```

```
const mongoose = require('mongoose');
const cors = require('cors');

// TODO: Import route modules
// - students
// - menu items
// - orders
// - analytics

const app = express();

// TODO: Read PORT from environment variables (use default if not provided)
// TODO: Read MongoDB connection URI from environment variables

// Middleware
// TODO: Enable CORS
// TODO: Enable JSON body parsing

// Root route
app.get('/', (req, res) => {
  // TODO: Send a JSON response indicating API is running
});

// Attach routes
// TODO: Mount student routes at /students
// TODO: Mount menu item routes at /menu-items
// TODO: Mount order routes at /orders
// TODO: Mount analytics routes at /analytics

// Database connection
// TODO: Connect to MongoDB using mongoose
// TODO: On successful connection, start the server
// TODO: Log server running message with PORT
// TODO: Handle MongoDB connection errors
```

2. Start the server and ensure it listens on a port (e.g., 3000).
3. Confirm the server runs without errors.

3.6 Connect to MongoDB

1. Decide whether to use:
    1. MongoDB Atlas, or
    2. Local MongoDB
2. Configure the MongoDB connection string.
3. Use Mongoose to connect to MongoDB.
4. Log success or failure messages.
5. Restart the server and verify successful connection.

```
> campus-food-api@1.0.0 start
> node server.js

[dotenv@17.2.3] injecting env (2) from .env -- tip: ⚙ enable debug logging with { debug: true }
✅ Connected to MongoDB
✅ Server running on http://localhost:3000
```

3.7 Attach Route Modules

1. Import route modules.
2. Mount routes with base paths:
    1. /students
    2. /menu-items
    3. /orders
    4. /analytics
3. Restart the server and ensure no routing errors.

4. Design the Data Model

You will implement **three MongoDB collections**.

4.1 Student Model

Fields:

1. name – string, required
2. email – string, required, unique
3. faculty – string
4. year – number

```
// models/Student.js
const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true
    },
    email: {
      // TODO: Define type as String
      // TODO: Make this field required
      // TODO: Make this field unique
      // TODO: Trim whitespace
    },
    faculty: {
      // TODO: Define type as String
      // TODO: Trim whitespace
    },
```

```
    year: {
      // TODO: Define type as Number
      // TODO: Set minimum value to 1
      // TODO: Set maximum value to 4
    }
  },
  {
    // TODO: Enable timestamps option
  }
);

const Student = mongoose.model('Student', studentSchema);

module.exports = Student;
```

4.2 Menu Item Model

Fields:

1. name – string, required
2. price – number, required
3. category – string (Rice, Beverage, Snack, etc.)
4. isAvailable – Boolean

```
// models/MenuItem.js
const mongoose = require('mongoose');

// TODO: Create a mongoose schema for MenuItem
const menuItemSchema = new mongoose.Schema(
  {
    name: {
      // TODO: Set type to String
      // TODO: Make this field required
      // TODO: Trim whitespace
    },
    price: {
      // TODO: Set type to Number
      // TODO: Make this field required
      // TODO: Set minimum value to 0
    },
    category: {
      // TODO: Set type to String
      // TODO: Trim whitespace
    },
    isAvailable: {
      // TODO: Set type to Boolean
      // TODO: Set default value to true
    }
  },
  {
    // TODO: Enable timestamps
  }
);

// TODO: Create the MenuItem model using mongoose.model()
// TODO: Export the MenuItem model
```

4.3 Order Model

Fields:

1. student – reference to Student
2. items – array of embedded objects:
    1. menuItem reference
    2. quantity
3. totalPrice – number
4. status – PLACED, PREPARING, DELIVERED, CANCELLED
5. createdAt – date

```
// models/Order.js
const mongoose = require('mongoose');

// TODO: Create a schema for order items (orderItemSchema)
const orderItemSchema = new mongoose.Schema(
  {
    menuItem: {
      // TODO: Set type as mongoose.Schema.Types.ObjectId
      // TODO: Add reference to 'MenuItem'
      // TODO: Make this field required
    },
    quantity: {
      // TODO: Set type as Number
      // TODO: Make this field required
      // TODO: Set minimum value to 1
    }
  },
  {
    // TODO: Disable _id for subdocuments
  }
);

// TODO: Create a schema for Order
const orderSchema = new mongoose.Schema({
  student: {
    // TODO: Set type as mongoose.Schema.Types.ObjectId
    // TODO: Add reference to 'Student'
    // TODO: Make this field required
  },
  items: {
    // TODO: Define this as an array of orderItemSchema
    // TODO: Make sure at least one item exists
  },
  totalPrice: {
    // TODO: Set type as Number
    // TODO: Make this field required
    // TODO: Set minimum value to 0
  },
  status: {
    // TODO: Set type as String
    // TODO: Restrict values using enum
    // TODO: Set default value to 'PLACED'
```

```
  },
  createdAt: {
    // TODO: Set type as Date
    // TODO: Set default value to current date
  }
});

// TODO: Create the Order model using mongoose.model()
// TODO: Export the Order model
```

5. Core CRUD Endpoints

5.1 Student Routes

1. POST /students – create student
2. GET /students/:id – get student by ID

```
// routes/students.js
const express = require('express');
const Student = require('../models/Student');

const router = express.Router();

// POST /students - create student
router.post('/', async (req, res) => {
  try {
    const student = new Student(req.body);
    const saved = await student.save();
    res.status(201).json(saved);
  } catch (err) {
    console.error('Error creating student:', err.message);
    res.status(400).json({ error: err.message });
  }
});

// GET /students/:id - get student by ID
router.get('/:id', async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);
    if (!student) {
      return res.status(404).json({ error: 'Student not found' });
    }
    res.json(student);
  } catch (err) {
    console.error('Error fetching student:', err.message);
    res.status(400).json({ error: 'Invalid student ID' });
  }
});

module.exports = router;
```

5.2 Menu Item Routes

1. POST /menu-items – create menu item

2. GET /menu-items – list menu items

```
// routes/menuItems.js
const express = require('express');
const MenuItem = require('../models/MenuItem');

const router = express.Router();

// POST /menu-items - create menu item
router.post('/', async (req, res) => {
  try {
    // TODO: Create a new MenuItem using request body
    // TODO: Save the menu item to the database
    // TODO: Return the saved item with status code 201
  } catch (err) {
    // TODO: Log the error message
    // TODO: Return a 400 response with error message
  }
});

// GET /menu-items - list all menu items
router.get('/', async (req, res) => {
  try {
    // TODO: Retrieve all menu items from the database
    // TODO: Sort items by createdAt in descending order
    // TODO: Return items as JSON
  } catch (err) {
    // TODO: Log the error message
    // TODO: Return 500 server error response
  }
});

// 7.1 Search Menu Items
// GET /menu-items/search?name=...&category=...
router.get('/search', async (req, res) => {
  try {
    // TODO: Extract name and category from request query
    // TODO: Create an empty filter object
    // TODO: If name exists, add partial case-insensitive match to filter
    // TODO: If category exists, add category to filter
    // TODO: Fetch filtered menu items and sort by name (ascending)
    // TODO: Return filtered items as JSON
  } catch (err) {
    // TODO: Log the error message
    // TODO: Return 500 server error response
  }
});

// TODO: Export the router
```

5.3 Order Routes

1. POST /orders – place order
2. GET /orders – list orders
3. GET /orders/:id – get order by ID
4. PATCH /orders/:id/status – update order status

5. DELETE /orders/:id – delete order

```
// routes/orders.js
const express = require('express');
const Order = require('../models/Order');
const MenuItem = require('../models/MenuItem');

const router = express.Router();

// Helper: calculate totalPrice from items
async function calculateTotalPrice(items) {
  // TODO: Extract menuItem IDs from items array
  // TODO: Retrieve menu items from database using $in
  // TODO: Create a price map (menuItemId -> price)
  // TODO: Calculate total price using quantity
  // TODO: Throw an error if an invalid menuItem ID is found
  // TODO: Return the calculated total
}

// POST /orders - place order
router.post('/', async (req, res) => {
  try {
    // TODO: Extract student and items from request body

    // TODO: Validate student and items
    // - student must exist
    // - items must be a non-empty array

    // TODO: Calculate total price using calculateTotalPrice()

    // TODO: Create a new Order with student, items, totalPrice and status

    // TODO: Save the order to the database

    // TODO: Re-fetch saved order and populate student and items.menuItem

    // TODO: Return populated order with status code 201
  } catch (err) {
    // TODO: Log error message
    // TODO: Return 400 response with error message
  }
});

// GET /orders - list orders (with pagination support)
router.get('/', async (req, res) => {
  try {
    // TODO: Read page and limit from query parameters
    // TODO: Set default values for page and limit
    // TODO: Calculate skip value

    // TODO: Fetch paginated orders
    // - sort by createdAt descending
    // - populate student and items.menuItem

    // TODO: Count total number of orders

    // TODO: Return pagination metadata and orders list
  } catch (err) {
    // TODO: Log error message
```

```
    // TODO: Return 500 server error response
  }
});

// GET /orders/:id - get order by ID
router.get('/:id', async (req, res) => {
  try {
    // TODO: Find order by ID
    // TODO: Populate student and items.menuItem
    // TODO: If order not found, return 404
    // TODO: Return order as JSON
  } catch (err) {
    // TODO: Log error message
    // TODO: Return 400 response for invalid order ID
  }
});

// PATCH /orders/:id/status - update order status
router.patch('/:id/status', async (req, res) => {
  try {
    // TODO: Extract status from request body
    // TODO: Define allowed status values
    // TODO: Validate status

    // TODO: Update order status using findByIdAndUpdate
    // TODO: Enable runValidators and return updated document
    // TODO: Populate student and items.menuItem

    // TODO: If order not found, return 404
    // TODO: Return updated order
  } catch (err) {
    // TODO: Log error message
    // TODO: Return 400 response with error message
  }
});

// DELETE /orders/:id - delete order
router.delete('/:id', async (req, res) => {
  try {
    // TODO: Delete order by ID
    // TODO: If order not found, return 404
    // TODO: Return success message
  } catch (err) {
    // TODO: Log error message
    // TODO: Return 400 response for invalid order ID
  }
});

// TODO: Export the router
```

6. Advanced Query Endpoints

```
// routes/analytics.js
const express = require('express');
const mongoose = require('mongoose');
const Order = require('../models/Order');
```

```
const MenuItem = require('../models/MenuItem');

const router = express.Router();

_
/**
 * 8.1 Total Amount Spent by a Student
 * GET /analytics/total-spent/:studentId
 */
router.get('/total-spent/:studentId', async (req, res) => {
  try {
    // TODO: Extract studentId from request parameters

    // TODO: Validate whether studentId is a valid MongoDB ObjectId
    // TODO: If invalid, return 400 error

    // TODO: Use MongoDB aggregation on Order collection
    //  - Match orders by studentId
    //  - Group by student
    //  - Calculate sum of totalPrice as totalSpent

    // TODO: If no orders found, return totalSpent as 0

    // TODO: Return studentId and totalSpent as JSON response
  } catch (err) {
    // TODO: Log error message
    // TODO: Return 500 server error response
  }
});

/**
 * 8.2 Top Selling Menu Items
 * GET /analytics/top-menu-items?limit=5
 */
router.get('/top-menu-items', async (req, res) => {
  try {
    // TODO: Read limit value from query parameters
    // TODO: Set default limit if not provided

    // TODO: Perform aggregation on Order collection
    //  - Unwind items array
    //  - Group by menuItem
    //  - Sum quantities
    //  - Sort by total quantity descending
    //  - Limit results

    // TODO: Populate menuItem details using MenuItem model
    // TODO: Format response to include menuItem and totalQuantity
```

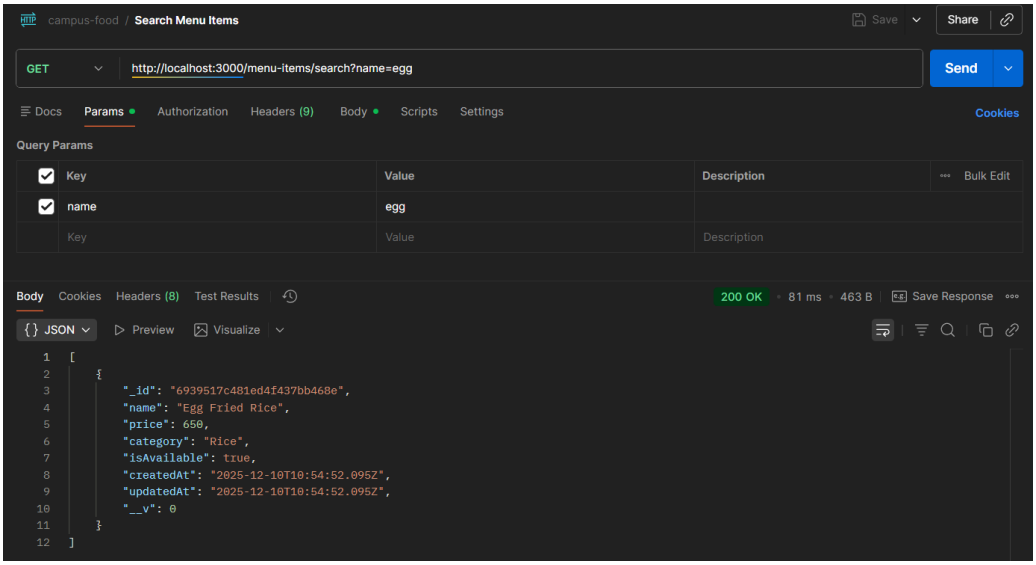
```
    // TODO: Return formatted result as JSON
  } catch (err) {
    // TODO: Log error message
    // TODO: Return 500 server error response
  }
});

/**
 * 8.3 Daily Order Counts
 * GET /analytics/daily-orders
 */
router.get('/daily-orders', async (req, res) => {
  try {
    // TODO: Use aggregation to group orders by date (YYYY-MM-DD)
    // TODO: Count number of orders per day
    // TODO: Sort results by date ascending

    // TODO: Format response to include date and orderCount
    // TODO: Return formatted result as JSON
  } catch (err) {
    // TODO: Log error message
    // TODO: Return 500 server error response
  }
});
// TODO: Export the router
```

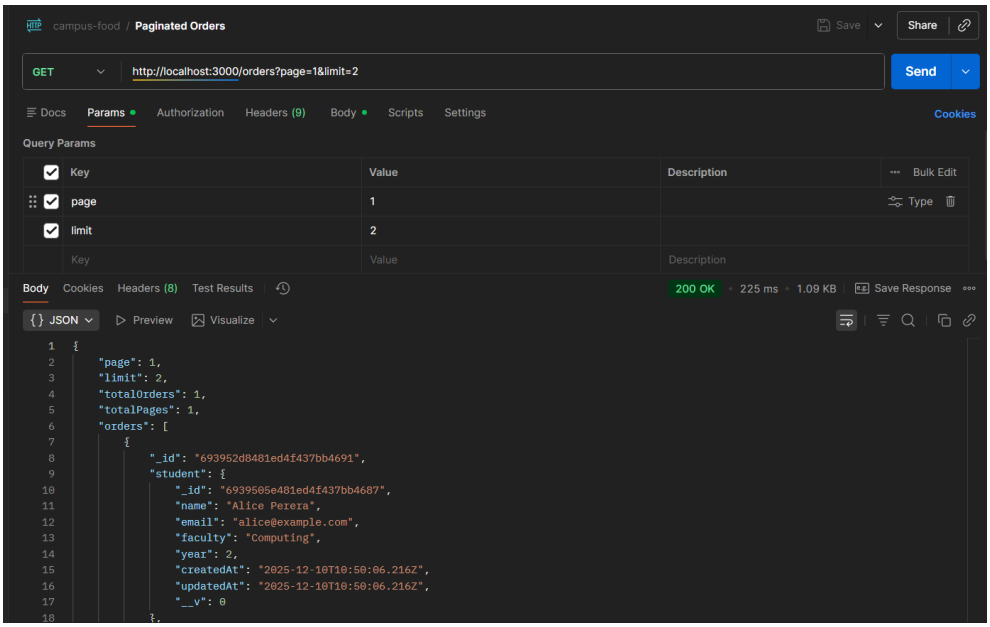
6.1 Search Menu Items

1. Endpoint: GET /menu-items/search
2. Query parameters:
    1. name – partial, case-insensitive
    2. category



6.2 Paginated Orders

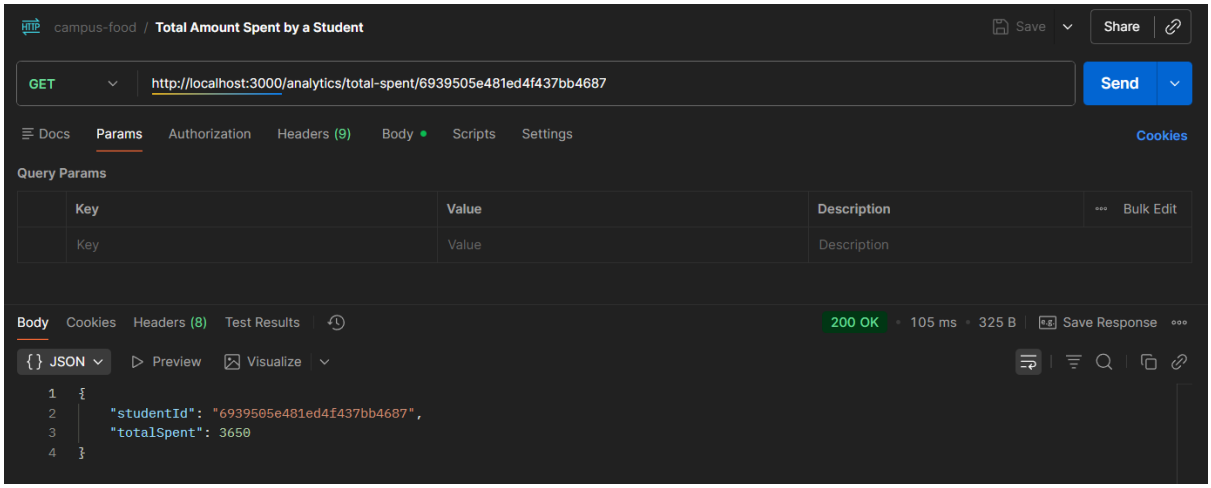
1. Endpoint: GET /orders
2. Query parameters:
    1. page
    2. limit
3. Return paginated orders and metadata.



7. Analytics & Aggregation Endpoints

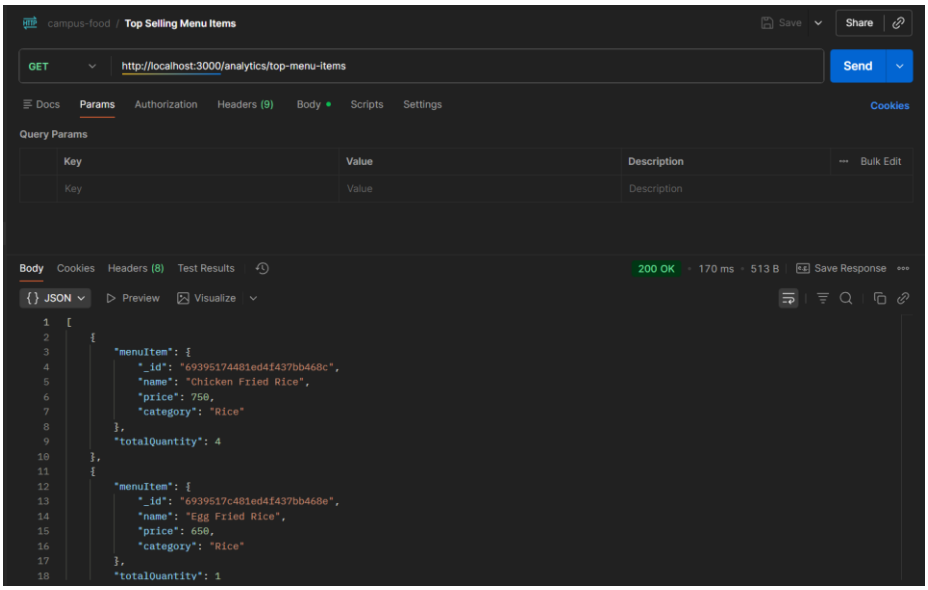
7.1 Total Amount Spent by a Student

- GET /analytics/total-spent/:studentId
- Aggregate orders to calculate total price per student.



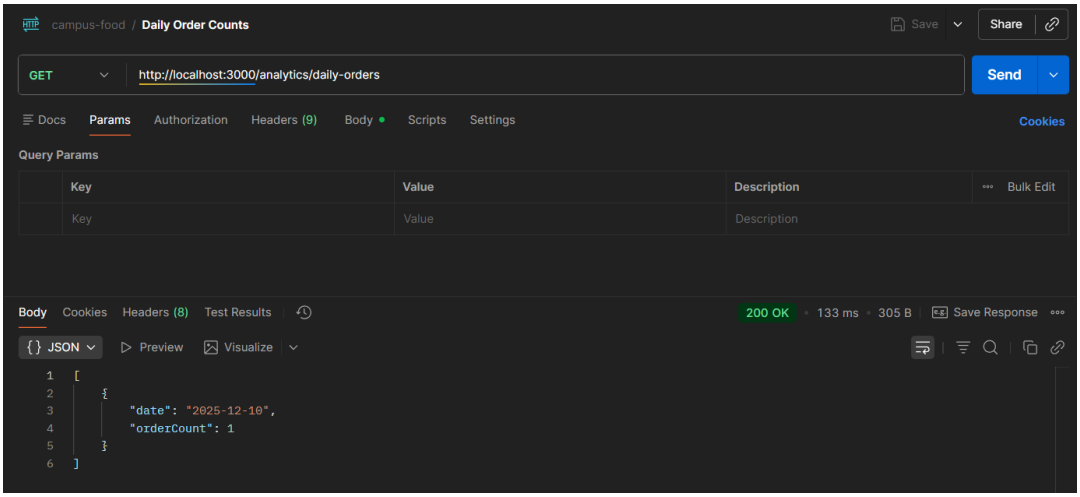
7.2 Top Selling Menu Items

- GET /analytics/top-menu-items
- Unwind items, group by menu item, sum quantities.



7.3 Daily Order Counts

- GET /analytics/daily-orders
- Group orders by date and count.



8. Testing with Postman

8.1 Create Initial Data

1. Create at least **3 students**.
2. Create at least **5 menu items**.
3. Place multiple orders.

8.2 Functional Testing

1. Verify CRUD operations.
2. Test search and pagination.

Validate aggregation results logically.